STUDY & EXAM PREPARATION GUIDE

OCP JAVA 17 & 21 PROGRAMMER CERTIFICATION FUNDAMENTALS

OFFICIAL COURSE COMPANION

CHAPTER 23

Using Modules

CHAPTER OVERVIEW

This chapter covers the following exam objectives: Define modules and their dependencies, expose module content including for reflection., Define services, providers, and consumers, Compile Java code, produce modular and non-modular jars, runtime i...

EXAM OBJECTIVES COVERED

- Define modules and their dependencies, expose module content including for reflection.
- Define services, providers, and consumers
- Compile Java code, produce modular and non-modular jars, runtime images

Chapter 23 Using Modules

Exam Objectives

- Define modules and their dependencies, expose module content including for reflection.
- Define services, providers, and consumers
- Compile Java code, produce modular and non-modular jars, runtime images
- Implement migration to modules using unnamed and automatic modules

23.1 Module Basics

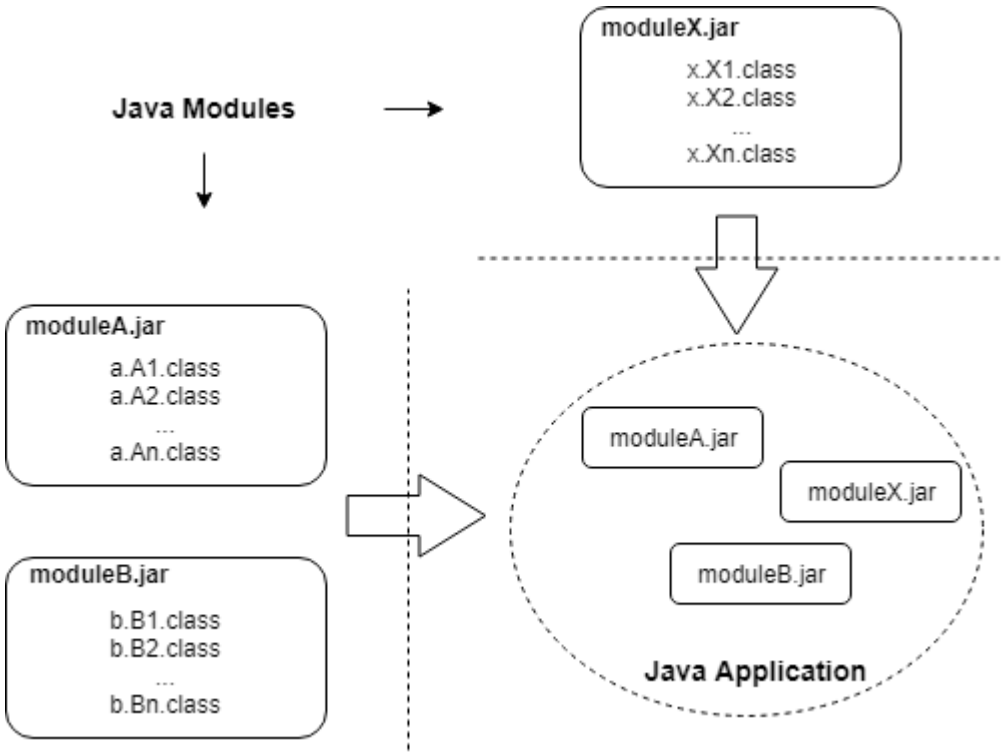
23.1.1 What are modules? ☝

In the Kickstarter for Beginners chapter, you saw how to package individual classes of an application into a jar file and how to run an application using that jar file. The jar file approach has been a great way to package multiple files into a single file and to deliver applications as well as class libraries to users.

However, Jar files are not without problems. Packaging classes into a jar file without a well thought out plan gives rise to unwieldy applications that are difficult to update and/or reuse. Jar files also make it difficult to use just a part of an application without having the whole set of jar files. If you are a Java beginner or haven't been involved in professional level Java development much, you may find it hard to believe that the jar file approach also led the Java community to something called "Jar Hell". Jar Hell is a term coined to describe hard to debug behavior of applications due to the presence of multiple versions of a class library on the classpath and/or due to the use of different versions of a library used by different parts of the applications. For example, what if an application uses Spring and Hibernate and these libraries require a different versions of the same JDBC driver or a logging library?

Although the JDK did not provide any out of the box solution to such issues, various Java community projects such as Ant, Maven, and Gradle have attempted to provide ways for managing these issues.

With Java 9, the Java language designers came up with a new approach called the Java Module System, also known as Project Jigsaw, to packaging Java classes that attempts to solve at least some of the problems mentioned above. It envisions a Java application (or a library) to be composed not of classes or jar files but of modules, where each module of an application clearly specifies what other modules it depends on and what features it provides to other modules. A module, in turn, is composed of classes. A module, therefore, is a much coarser grained component of an application than a class. In other words, classes are packaged in a jar file to form a module, and modules are mixed and matched at run time to form an application. The following diagram illustrates how classes and modules come together to make an application.



The above diagram shows three jar files named `moduleA.jar`, `moduleB.jar`, and `moduleX.jar`. These module jar files are used together to create an application. Note that an application is just a loosely bound collection of modules and is not packaged

as a jar file itself. You will soon see that an application is created by specifying the required modules from different teams, groups, or companies on the command line at run time.

It is not like professional application teams were not already grouping independent parts of an application into separate jar files. Indeed, even before the advent of modules, Java applications were composed of jars collected from various teams. But the process of grouping the classes and, more importantly, documenting what a jar file requires and provides, varied from team to team and from build tool to build tool. Java 9 takes the best of the practices prevalent in the industry and formalizes them into the Module System.

Besides formally specifying a structural design for applications, Java 9 also includes new tools and enhances existing tools to help developers package their code as modules.

Package vs Module ☝

You may be wondering at this point where packages fit into this picture. After all, we have been using packages to organize our Java code since ages. Well, packages still play the same role. They are still used to organize our code but at a finer level.

Conceptually, a package creates a namespace where all of the code required to implement a particular functionality resides. It is a means to reduce the size of the code contained in a class file by separating the logic into multiple closely related classes. This helps in reading, understanding, and navigating the code. But this is from the perspective of the developer of the functionality and not from the perspective of the users of that functionality. The users of that functionality are not interested in knowing how the code is organized. They are interested in knowing what it takes to use that functionality. Users simply want to use the functionality as a black box. Unfortunately, packages do not provide this information. Developers have been relying on informal methods such as naming jar files according to their own conventions and adding release notes to deliver this information.

This is where modules come in. A module provides a way to deliver functionality in a single artifact that includes not just the code but also the information on the services that the code provides and any dependencies that it may have.

Thus, if you are a developer of some functionality, you would still organize your code into packages. But you would then also package all of your classes (and any other resources such as image files, xml files, or property files) that are used to implement this functionality into a single deliverable with a description of your functionality and its dependencies in a prescribed format in the form of a module. In other words, modules are not an alternative to packages but are an addition on top of packages.

The Java module system is a fairly large topic with dedicated books written to cover it. However, the OCP Java exam focuses only on the basics of the Java module system.

23.1.2 Declaring a module ☝

Since a module is a logical unit of functionality from the users' perspective, it is best to first identify the functionality that we want to deliver as a module. For example, let us say we are developing an application for finance and one of its functions is to compute simple interest. We want to deliver this simple interest calculator as a module.

Naming a module ☝

A module name follows the same rules and conventions that you saw for naming packages and classes earlier. A valid package name would therefore, be a valid module name. In short, the name must be a valid Java identifier (so, special characters such as dash and slash are out but underscore and dot are in) and it should follow the reverse domain name pattern to avoid name clash. Ideally, the name should be descriptive enough to tell the user the purpose and/or functionality of the module instead of being too generic such as `tools` or `utils`. So, `com.abc.finance.calculators.simpleinterest` would be a good name for our module. However, for the purpose of this chapter, I am going to name the module

as just `simpleinterest` to reduce clutter and to make it easy to try out the code that I am going to present here. Let us also decide that the FQCN of our class will be `simpleinterest.SimpleInterestCalculator` instead of `com.abc.finance.calculators.simpleinterest.SimpleInterestCalculator` for the same reason. Note that there is no relation between the module name and the package name(s) of the class(es) contained in that module. However, since both follow the reverse domain name convention, they may very well be the same.

The exam expects you to be able to identify valid and invalid module names. As explained above, the rules for naming a module are the same as those for naming a package. Thus, for example, the following are valid module names: `a` (length is not important), `_m` (may start with and contain _), `n$` , (may start with and contain $), `x1` (may contain a number) and `a.b` (may contain dots) but the following are invalid: `1x`, (cannot start with a number), `a-b`, (cannot contain hyphens), and `a..b` (cannot contain consecutive dots).

The module descriptor ☝

Remember I talked about the lack of any formal way to describe a package? This is where a module differs from a package. Module descriptor is the formal way to describe a module. Every module must have a module descriptor that specifies the name of the module, what it provides, and what it requires. A module descriptor is nothing but a file by the name `module-info.java` . Yes, the extension of the file is `java` . It is a Java file that will be compiled to generate a class file by the name `module-info.class` . When we talk of a module, it is this descriptor that we generally refer to. Indeed, the module itself is a black box and the module descriptor is all the user should worry about. In our case, the contents of the module descriptor will be as follows:

```
//in file module-info.java
module simpleinterest{
}
```

There isn't much in this module descriptor. All it says is that this is a module named `simpleinterest` . But it is a valid module descriptor nonetheless and it illustrates

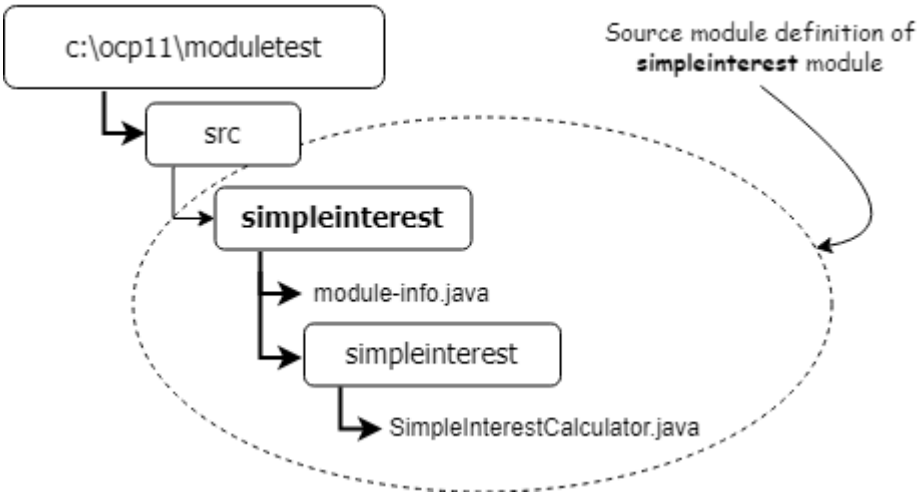
the basic syntax for defining a module. You have the module keyword and the module name, followed by the opening and closing braces. We will be adding more information in this descriptor as we go along. At this time, however, it is a good start!

The `module-info.java` file cannot be empty.

23.1.3 Directory structure of a module ☝

The directory structures for the sources and for the compiled classes of a module are almost exactly the same as that of a package. The only difference is that while a package may reside in any folder, a module resides in a folder by the same name as the name of the module. This rule applies to the source files as well as the class files. If the module directory contains the source code, it is called "**source module definition**" and if the module directory contains compiled classes, it is called "**module definition**".

Thus, in our example, the source code and compiled classes of our module must reside in folders named `simpleinterest`. The following image shows the directory structure that I will use to develop and execute the code for our module:



My base directory is `c:\ocp21\moduletest`. This is the directory where I am going to develop and run all the code presented in this chapter. If you are using Linux/MacOS, you might want to create a `moduletest` directory under `/home/<username>`. (Remember to flip `\` to `/` in directory paths if you are using Linux/MacOS.)

Under the base directory, I have a `src` directory, where I will keep the source code for all of my modules. Since this particular module is named `simpleinterest`, I have a directory by the same name under `src`. The directory tree rooted at `simpleinterest` is therefore, the source module definition of the `simpleinterest` module. Furthermore, since our `SimpleInterestCalculator` class belongs to the `simpleinterest` package, I have kept `SimpleInterestCalculator.java` under `simpleinterest\simpleinterest`.

Note that it is not necessary to follow the package structure for organizing Java source files. You may keep all your Java source file(s) directly under the module root folder as well. However, organizing the source code as per their packages has a couple of advantages. Besides making the source code easy to navigate, it is understood well by the Java compiler. The compiler can automatically locate and compile the code present in a source module definition.

The contents of `SimpleInterestCalculator.java` are as follows:

```
package simpleinterest;
public class SimpleInterestCalculator{
   public double calculate(double principal, double rate, double
time){
      return principal*rate*time;
   }
   public static void main(String[] args){
     System.out.println(new SimpleInterestCalculator().calculate(100, .05,
2));
   }
}
```

23.2 Compiling and running a modular program

23.2.1 Compiling a module ☝

Now that our directory structure and the code base for the module is ready, let us compile it. Open a command/shell prompt, `cd` to the `c:\ocp21\moduletest` directory and run the following command:

```
c:\ocp21\moduletest>javac -d out --module-source-path src --module simpleinterest
```

The above command uses three command line switches: `-d`, `--module-source-path`, and `--module`.

You have used the `-d` switch before to direct the output of the Java compiler to a particular directory. It works the same way here. So, `-d out` will cause the compiler to produce its output in the `out` directory. The compiler will create the `out` directory if it does not already exist.

The `--module-source-path` switch tells the compiler the location of the source module definition. Since we have kept our source code in a directory under the `src` directory, that is what we have specified here as well. Observe the double dash in this switch.

Since the name of the module is **simpleinterest**, the compiler will create a `simpleinterest` directory under `out`. The `out\simpleinterest` directory is the `simpleinterest` module's **"root directory"** and the tree rooted under `simpleinterest` is the **module definition** of the `simpleinterest` module.

The compiler will save the class files of this module inside `out\simpleinterest` as per the package driven directory structure of the classes of this module. For example, since `SimpleInterestCalculator` class is in `simpleinterest` package, the

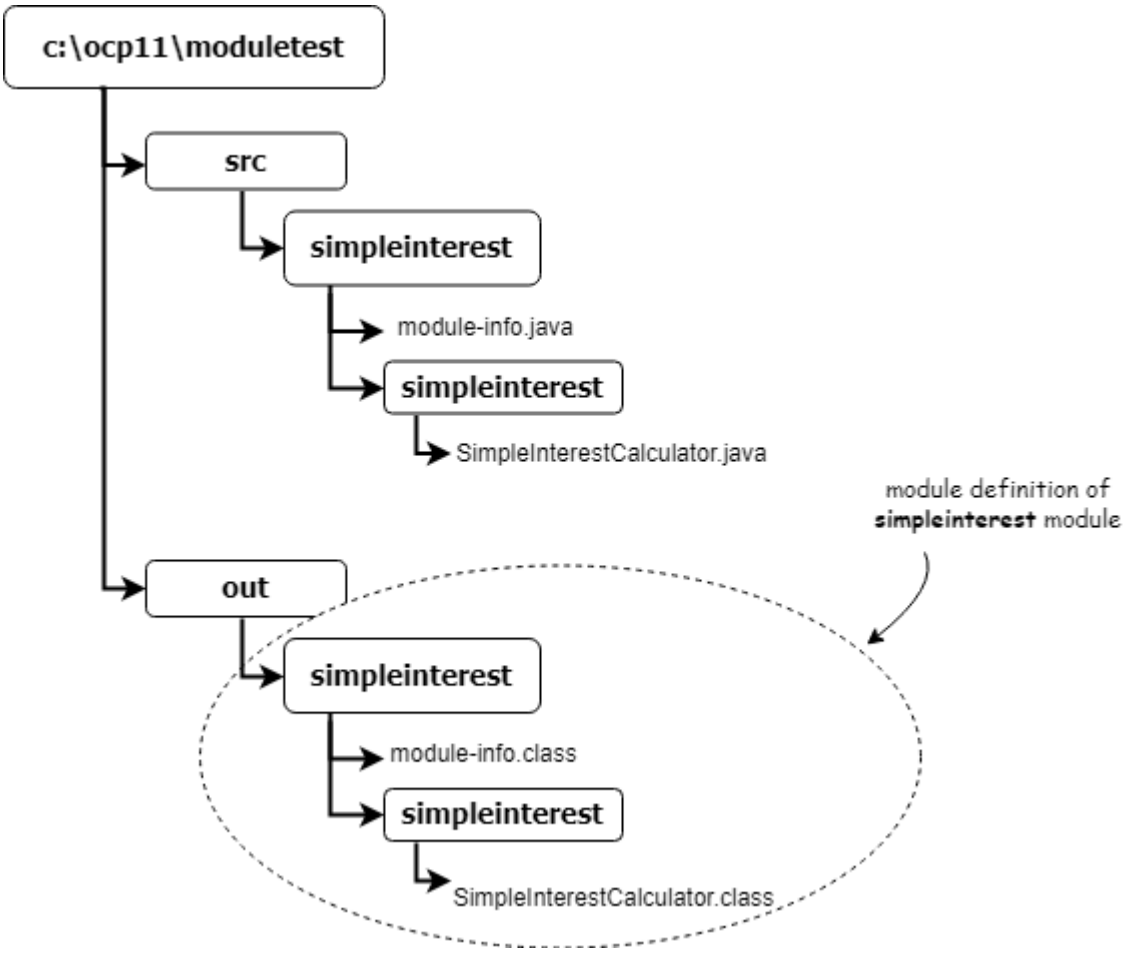
compiler will put `SimpleInterestCalculator.class` under `out\simpleinterest\simpleinterest` directory.

Note that I am using relative paths instead of absolute paths (i.e. `out` instead of `c:\ocp21\moduletest\out`). Relative paths are relative to the current directory, and since our current directory is `c:\ocp21\moduletest`, `out` resolves to `c:\ocp21\moduletest\out`. You may use absolute paths and relative paths interchangeably in these commands but it is better to use relative paths because they will work even if your base directory something different from `c:\ocp21\moduletest`.

The `--module` switch specifies the name of the module that we want to compile. The compiler tries to locate this module in the paths specified in `module-source-path`. In this case, we want to compile the `simpleinterest` module. Again, observe the double dash.

Based on the above options, the compiler will look for `module-info.java` under `src\simpleinterest` directory and will compile all the Java files under all the directories that are present inside `src\simpleinterest` directory.

Upon successful execution of the above command, you should see the directory structure as shown below:



Observe the directory structure generated by the compiler under `out\simpleinterest`. It is the same as what you get after compiling non-modular Java source files.

I suggest that you play around with the above command by changing the particulars of this set up. For example, rename the module source directory to something else or move the Java source file to another location within the module source directory and observe the error message generated by the compiler.

Compiling individual module source files ☝

It is possible to compile module source files without using the `--module` option by

listing the files (either individually or using a wildcard) that you want to compile. For example, the following old style command will produce the same result as produced by the command used above:

```
c:\ocp21\moduletest>javac -d out\simpleinterest src\simpleinterest\module-info.java src\simpleinterest\simpleinterest\SimpleInterestCalculator.java
```

Observe that you have to specify the module root directory component `simpleinterest` in the output path explicitly. Without this, the compiler will save the output under `out` instead of `out\simpleinterest`. Listing individual files for compilation is not a preferred way to compile a module and is seldom used. You will not be tested on this in the exam either. For the purpose of the exam, you are expected to know how to compile a module using the `--module` switch only.

23.2.2 Running a module ☝

Running a module means executing the `main` method of a class contained in that module. Thus, the class that we want to execute must have the standard main method. The following command shows how to run our `simpleinterest` module that we just compiled.

```
c:\ocp21\moduletest>java --module-path out --module simpleinterest/simpleinterest.SimpleInterestCalculator
```

This command uses two switches: `--module-path` and `--module`.

The `module-path` switch is used to tell Java the location of the module definition. This is the directory where the module's root directory is located or, if the module is packaged as a jar file, module-path is the directory where that jar is located. In our case, we have specified `out` in `module-path` because our module's root directory, i.e., `simpleinterest`, is located in `out`.

The `module` switch is used to tell Java the name of the module that we want to execute. Observe the value specified for `--module`. It is in the format `<module name>/<main class name>` because, as of now, our module is present in an exploded format and does not contain any information about the main class. So, we need to tell the JVM about the class that we want to run as well. Once we package our module into a jar, we won't need to specify the main class on the command line.

Packaging a module ☝

The process of packaging classes of a module into a jar file is the same as the one we saw for packaging classes into a regular jar. The only thing special about a module jar is that the name of the jar is, by convention, the same as the name of the module (with `.jar` extension, of course). Inside the jar file, class files must still exist in their package driven directory structure just like before. Thus, the name of the jar file corresponds to the module directory and the organization of the files inside the jar corresponds to the organization of the files inside the module directory. Since a module also has a `module-info.class` in its root folder, the jar file must also have this class in the root folder. Note that the requirement of having the `module-info.class` in the root folder of the jar implies that it is not possible to package multiple modules in the same jar. In other words, you can have only one module per jar.

Since a modular jar (including its name) mimics the organization of a module, a modular jar is also considered as the **"module definition"** of a module.

Here is the command that will package our module into a jar file:

```
c:\ocp21\moduletest>jar --create --file simpleinterest.jar --main-class simpleinterest.SimpleInterestCalculator -C out\simpleinterest .
```

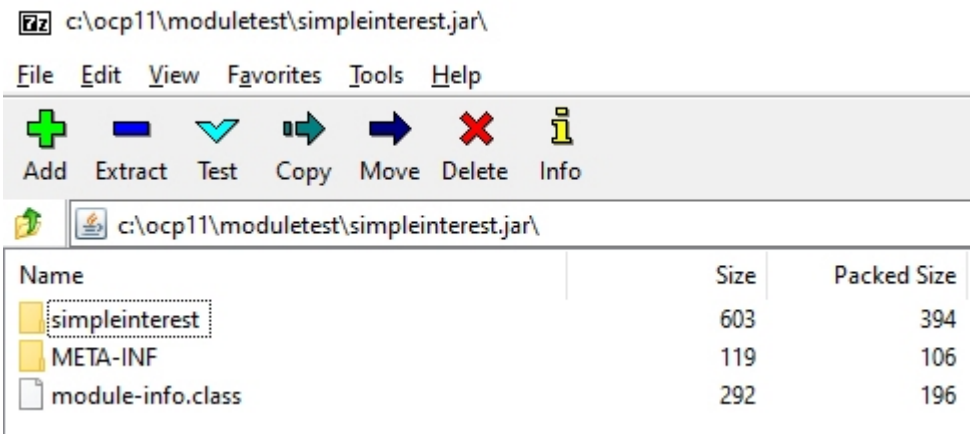
This should create `simpleinterest.jar` in the `moduletest` directory.

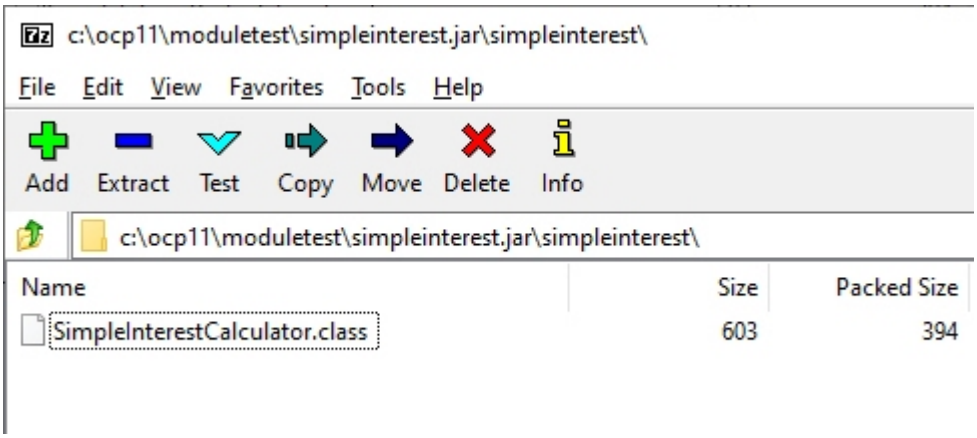
The `--create` switch tells the jar tool that we want to create something and the `--file` and the subsequent value specify the jar file that we want to create. The `--`

`main-class` switch makes the jar tool add a `Main-Class` entry in the resulting jar file's manifest.

The `-C` switch needs some explanation. We want the jar file to mimic the directory structure of the module, which means the structure inside the jar file should be the same as the structure inside the `out\simpleinterest` directory. But we are executing the command from the `moduletest` directory. There are two extra levels under `moduletest` (`out` and `simpleinterest`) that we don't want reflected in the jar file. The `-C` option is to make the jar tool change its working directory before adding files to the jar. We want it to step down to `out\simpleinterest` directory and then include the files from there.

The dot at the end of the command means that we want to include everything in its current directory (which is now `out\simpleinterest` due to the use of the `-C` switch) to be included in the jar file. The following two images show the contents of `simpleinterest.jar` created by the above command. (I have used the free version of **7zip** application to view the contents of the jar file.) The first image shows the contents at the root level and the second image shows the contents of the `simpleinterest` folder:





Running this module is now a piece of cake:

```
c:\ocp21\moduletest>java --module-path . --module simpleinterest
```

Observe that, since we added the `Main-Class` attribute in jar's manifest, we are able to "run" the module.

Another important point that you should observe in the above two images is that the location of `SimpleInterestCalculator.class` inside the module jar is exactly the same as it would have been had the class file been placed in a regular jar, i.e., under `<root>/simpleinterest`. This means, any application that doesn't understand modules can use a modular jar as if it were a regular jar. The only extra thing in a modular jar is the `module-info.class` file in the root folder, which will be ignored by a non-modular application. For example, you can run `SimpleInterestCalculator` in the old fashioned way with this command:

```
c:\ocp21\moduletest>java -classpath .\simpleinterest.jar
simpleinterest.SimpleInterestCalculator
```

You need not worry too much about the various switches used in the jar command. You won't be tested on how to create a jar file in the exam. However, executing a module packaged in a jar file is on the exam. So, it would be good if you understand how a modular jar file is created.

Understanding module-path ☝

Another interesting thing in the above command is the value that we have specified for `module-path`. We have specified only a dot for the `module-path` because our module jar is in the current directory. A module-path contains all the locations where you want the JVM to search for module definitions. While trying to load a module, the JVM looks for that module in all the locations specified in the `module-path`. It expects to find exactly one of the two things - a directory with the same name as the name of the module or a jar file containing `module-info.class` for that module. The name of the jar file is not important because the JVM takes the name of the module from the `module-info.class` contained in the jar file. The same module should not be present more than once on the `module-path`, otherwise the JVM will complain.

In the command used above, while searching for the module named `simpleinterest`, the JVM will look for a jar file that contains `module-info.class` with the module name as `simpleinterest` at the root or a directory by the name of `simpleinterest` containing appropriate `module-info.class`.

Compare this with the `-classpath` switch, which requires all jar files that you want on the class path to be listed explicitly instead of requiring just the directory that contains the jar file(s).

23.3 Enabling access between modules

23.3.1 Using exports and requires directives ☝

Let us improve the design of our `simpleinterest` module a little bit. It is a good design practice to define functionality in the form of an interface and let a class implement that interface. This ensures that the user of the functionality is able to replace one implementation with another without much impact on their code. Java modules allow us to take this technique even further by defining functionality separately in a module of its own. Separating the interface and the class that

implements that interface into separate modules allows us to build an application by mixing and matching modules without the need to bundle classes that are not required for the application.

In our case, we will create a `calculators.InterestCalculator` interface in `calculators` module. We will then have our `SimpleInterestCalculator` class implement this interface. Here is the interface definition:

```
//in file calculators\calculators\InterestCalculator.java
package calculators;
public interface InterestCalculator{
   public double calculate(double principal, double rate, double
time);
}
```

and the following is the module descriptor for the new module:

```
//in file calculators\module-info.java
module calculators{
   exports calculators;
}
```

The only thing new in this module definition is the `exports` clause. A module is an insular unit of packages. Members of a module are normally not accessible to code belonging to other modules. To make a package eligible to be accessible to other modules, the package must first be "exported" explicitly in the module descriptor. Making something eligible to be accessible only through an explicit `exports` clause ensures that code from other modules does not access code that is internal to this module inadvertently. Note that I am talking about packages instead of classes because only packages can be exported and not individual classes. When you export a package, all of the public types contained in that package are made eligible to be accessible by other modules. In module parlance, accessing a module is called "reading" the module. So, the `exports` clause in the above code makes the `InterestCalculator` interface eligible to be read by classes in other module. I have used the phrase "eligible to be read" because an exports clause is only one of the **two requirements** that must be fulfilled for a module to access classes from another

module. The second requirement is the "requires" clause, which I shall introduce shortly.

The directory structure of this module is as per the source module definition format explained earlier. The `c:\ocp21\moduletest\src\calculators` directory is the source directory for this module. It contains `module-info.java` and `calculators\InterestCalculator.java` files. Let us run the following command to compile this module:

```
c:\ocp21\moduletest>javac -d out --module-source-path src --module calculators
```

This should create appropriate output in the `out` directory.

Let us now move our attention to the `simpleinterest` module. Since we want `SimpleInterestCalculator` to implement `InterestCalculator` interface, we need to first make our intention clear by stating that our `simpleinterest` module "requires" access to the `calculators` module by modifying `simpleinterest`'s module descriptor as follows:

```
module simpleinterest{
   requires calculators;
}
```

The `requires` clause is the counterpart of the `exports` clause. A `requires` clause makes the other module "readable" in this module. This is also known as **module readability**. In other words, all the packages exported by the other module can be accessed by a module that declares its dependency on that module using the `requires` clause.

The purpose of having a `requires` clause is to make the dependencies of a module explicitly clear to the users. The users of a module need not go through the `import` statements of each class of a module to know which other modules does this module depend on. All they need is to check the `module-info`.

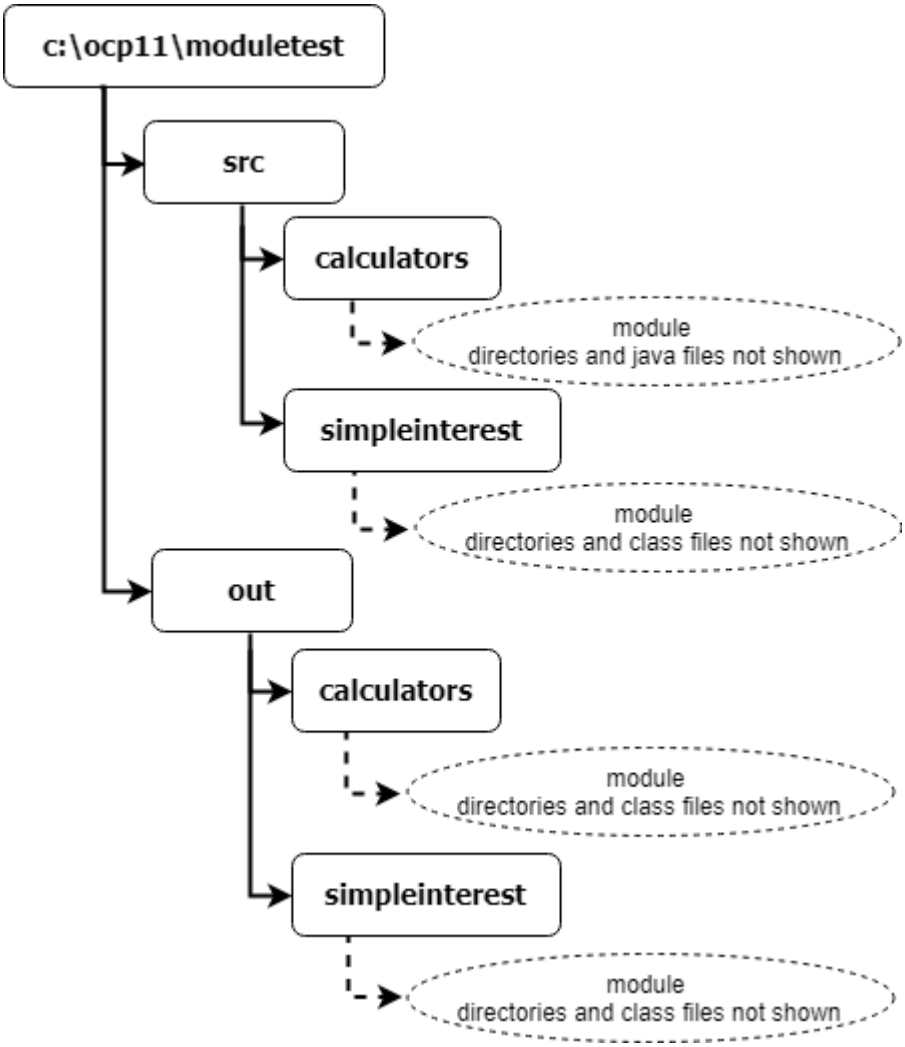
We can now modify `SimpleInterestCalculator.java` as follows:

```
package simpleinterest;
import calculators.InterestCalculator;
public class SimpleInterestCalculator implements
InterestCalculator{
   public double calculate(double principal, double rate, double
time){
      return principal*rate*time;
   }
   public static void main(String[] args){
      InterestCalculator ic =  new SimpleInterestCalculator();
      System.out.println(ic.calculate(100, .05, 2));
   }
}
```

The following command can be used to compile this module:

```
c:\ocp21\moduletest>javac   -d   out   --module-source-path   src   --module
simpleinterest
```

After the compilation of this module, our directory structure looks like this:



We can now execute the `simpleinterest` module as we did before using the following command:

```
c:\ocp21\moduletest>java          --module-path          out          --module
simpleinterest/simpleinterest.SimpleInterestCalculator
```

23.3.2 Using qualified exports ☝

The `exports` clause opens up a package to access from all other modules. Once a package is exported, you cannot stop any other module from accessing it. It is similar

to making a member of a class `public`, and just like the `public` modifier, the `exports` clause makes a module less encapsulated.

The Java module system allows you to fine tune access to a module only to specific modules using a variation of the `exports` clause. It is called the **exports-to** clause or the qualified exports clause. The following is how it is used:

```
module <modulename>{
   exports <packagename> to <modulename(s)>;
}
```

For example, in the following module-info, the `com.abc.datatransfer` package is being made accessible only to `db` and `service` modules:

```
module datatransfer{
   exports com.abc.datatransfer to db, service;
}
```

The modules for which a package is made readable are called "friendly" modules. Java standard library uses this feature heavily for its internal modules. For example: the `java.base` module of the JDK has several packages which are supposed to be used only by a selected few internal modules. It uses qualified exports to achieve the same. Here is a partial code snippet from `java.base` module's module-info:

```
module java.base {
   ...
   exports jdk.internal.logger to
      java.logging;
   exports jdk.internal.perf to
      java.desktop,
      java.management,
      jdk.jvmstat;
   ...
}
```

There is no particular order in which unqualified and qualified exports have to appear in a module-info. Conventionally, however, qualified exports are listed after all the unqualified exports.

Using qualified exports provides protection from creating inadvertent dependencies on packages that are not meant to used by everyone. Therefore, it is a good idea to use qualified exports as much as possible. Unqualified exports should be used only when you are absolutely sure that it is ok for the package to used by any other module.
It is a compile-time error if the package being exported using the `exports` or the `exports-to` clause does not exist in the current module. It is fine if the package listed in the `to` clause does not exist.

23.3.3 Using transitive dependencies ☝

A module `A` may read another module `B` by using a `requires` clause in its module-info. But what if the module `B` reads yet another module `C`? Does that mean module `A` reads the module `C` as well? Normally, no. Module dependencies are not transitive and if module `A` specifies that it requires only module `B`, it cannot read module `C` just because `B` reads `C`.

Now, let's see how this pans out in the following situation. What if a class in module `A` needs to invoke a method of a class in module `B`, whose return type is defined in module `C`? Something like this:

```
module ui{
   requires hr;
}
module hr{
   requires valueobjects;
   exports hrservice;
}
```

```
//code appearing in a class in ui module
HRService hrService = new HRService(); //HRService is defined in hr module
Employee e = hrService.getEmployee(employeeId); //Employee is defined in valueObjects module
```

Since the `ui` module does not have a `requires valuesobjects;` clause, the `ui` module cannot access the `Employee` class from the `valuesobjects` module. Thus, the above code will fail to compile.

The obvious solution is to add a `requires valuesobjects;` clause in the `ui` module. But this solution has a serious problem. What if the `hr` module has several `requires` clauses? How would the `ui` module know which of the `hr` module's `requires` clauses should be added to its `module-info`? Only multiple compilation failures can make this information known to the `ui` module!

This is where the **requires transitive** clause comes into picture. Since the `hr` module knows that whichever module requires `hrservice` package, will require classes in the `valueobjects` module as well, it documents this information using the `requires transitive` clause, like this:

```
module hr{
   requires transitive valueobjects;
   exports hrservice;
}
```

This basically says that if you require the `hr` module, then you also require the `valueobjects` module. But more than merely documenting this fact, the `requires transitive` clause also eliminates the need for adding an extra `requires valuesobjects;` clause in the `ui` module. The `requires hr;` clause in the `ui` module automatically makes all the modules transitively required by `hr` module, readable to the `ui` module. This is called **"implied readability"**.

Duplicate import statement vs duplicate requires/export clauses

Having duplicate `import` statements in a Java source file doesn't make any sense but Java has, historically, permitted duplicate `import` statements. However, Java does not permit having duplicate `requires` clauses in a `module-info`. Therefore, the following `module-info.java` will not compile:

```
module hr{
   requires valueobjects;
   requires valueobjects; //duplicate, not allowed
}
```

Requiring the same module in a `requires` clause and in a `requires transitive` clause is not allowed either. Thus, the following will not compile:

```
module hr{
   requires valueobjects;
   requires transitive valueobjects; //duplicate, not allowed
}
```

Similarly, duplicate `export` and `exports to` clauses are not permitted either.

Cyclic/Circular dependency

Java does not allow modules to circularly depend on each other. For example, if module `a` contains a `requires b;` clause, then module `b` cannot have a `requires a;` clause. Similarly, even an indirect circular dependency (i.e. a requires b, b requires c, and c requires a) is not allowed either.

Circular dependencies within classes and packages are allowed though.

23.3.4 Using opens/opens-to directives ☝

If you really do want the packages of a module to be readable by other modules, the

`exports` and `exports-to` directives are sufficient. But, as explained earlier, exporting a package is like letting the cat out of the bag. It is hard to put it back. Exporting a package allows other modules to use that package and once other modules start using it, you can't "un-export" it, i.e., remove it from the list of exported packages in your module-info, without breaking their code.

Large corporate projects often face this problem where one group uses a library developed and maintained by another group for their application. If the group providing the library decides one day to un-export a package in the newer version of that library, other groups that depend on that library will complain. As you will learn later, this could also happen when a group decides to modularize their library and determines a particular package shouldn't really be depended upon by other groups.

The Java Module system provides a way out of this quandary by making it possible for existing applications to use the un-exported packages at run time but prevent them from using the un-exported packages at compile time. This means, the applications that depend on the un-exported packages will continue to run as before but they will eventually have to fix their code to remove the dependency on the un-exported packages if they want their code to compile without errors.

This is done using the `opens`, `opens-to`, and `open` directives.

The `opens` directive ☝

The opens directive lets other modules access the `public` and `protected` types of this package at run time, but not at compile time. For example:

```
module simpleinterest{
  //exports simpleinterest; //commenting it out because we don't want anyone to use this package anymore
  opens simpleinterest; //lets any code that depends on this package to run
}
```

If the `simpleinterest` module exported the `simpleinterest` package, other modules may have been using the `SimpleInterest` class directly. Replacing it with the `opens` clause ensures that their application does not fail at run time but also forces them to stop using `SimpleInterest` class directly in their code.

The `opens-to` directive ☝

The `opens-to` directive is a qualified opens directive that works just like the qualified `exports` directive. It allows only the listed modules to have a runtime dependency on this package.

The `open` directive ☝

If you want to open all `public` and `protected` packages of a module, then you may apply the `open` directive to the module declaration itself instead of applying the `opens` directive to each package individually. For example:

```
open module simpleinterest{
```

```
//opens simpleinterest; //can't have any opens directive if the
module is open.
}
```

Opening a package for Reflection ☝

Besides providing a stepping stone while modularizing a library, opening a package in the manner described above also allows members of a module to be accessible through Reflection. Reflection is an advanced topic and is, fortunately, not on the OCP exam. You should definitely read more about it from other sources whenever you have time but n the context of Modules, all you need to know about Reflection is that it allows us to use classes at runtime that we didn't know about at compile time. By

"use", I mean creating objects of the class and invoking methods on it. Reflection can also be used to invoke methods and access fields of a class that have been declared `private`.

Since Reflection is quite a powerful and intrusive tool, the Java module system puts severe restrictions on its operation. In fact, Java **does not allow** modular code to be intruded upon through Reflection by code from other modules **unless** a module allows such access explicitly using the `open` or `opens` / `opens-to` directives in its `module-info`.

So, in a nutshell, the `open` directive allows **all members**, irrespective of their access modifiers, of the module to be accessible through the Reflection API. The `opens` directive makes only the members of the specified package to be accessible through reflection and the `opens-to` directive allows reflective access to the package mentioned in the `opens` clause only to the packages listed in the `to` clause.
Unlike the `exports` / `exports-to` directives, it is allowed to open a package that does not exist.

But just like `exports` / `exports-to` directives, it is NOT allowed to have duplicate `opens` / `opens-to` clauses or to list a package twice.

23.4 Advanced module compilation and execution ☝

Seeing all the new module related command line switches feels overwhelming at first. But once you understand the underlying theme behind these options, you will realize that they are not much different from the `-classpath` (and `-sourcepath`) options that you have used so far. The m`odule-path` and `module-source-path` switches

are to a modular project what `classpath` and `sourcepath` switches are to a regular project. Earlier, you used to specify just the FQCN of the main class while launching an application, now, you specify the module name using the `--module` option. That is basically it.

I have observed that many students get confused due to the fact that `classpath` and `module-path` are not mutually exclusive. It is possible, and sometimes necessary, to use both the switches simultaneously in a command while compiling as well as executing a module. They are used together in projects that make use of modular as well as non-modular libraries.

Another cause of confusion is the presence of multiple switches that mean and do the same things. For example, `--module-path` is the same as `-p` and `--module` is the same as `-m`.

In this section, I will summarize all the options that you need while compiling and executing a module.

23.4.1 Compiling multiple modules at once ☝

In the previous example, we compiled the `calculators` and `simpleinterest` modules separately one after another. We could have easily compiled both of them together at the same time by listing both of them in the `--module` switch:

```
c:\ocp21\moduletest>javac -d out --module-source-path src --module calculators,simpleinterest
```

Observe that the module names are separated using a comma. In fact, Java will compile the `calculators` module even without us specifying it explicitly in the `--module` switch because `javac` will notice that `simpleinterest` requires `calculators` and so, it will compile `calculators` first. However, this is possible only when the source code of the module is organized correctly in the package driven directory structure. If the source files are organized differently, `javac` will not be able

to locate them on its own.

Using a module without source code ☝

While developing a module, you may be required to use third party modules whose source code is not available. For example, what if the `calculators` module was developed by another team and you just had a `calculators.jar` for it?

Well, we can use the `--module-path` switch for telling `javac` the location of a module just like we used it while executing a module. Let's say we put `calculators.jar` in `c:\ocp21\moduletest\modulejars` directory. We can now use the following command to compile the `simpleinterest` module:

```
c:\ocp21\moduletest>javac --module-path modulejars -d out --module-source-path src --module simpleinterest
```

Observe that we didn't specify the actual jar name in the module path (although we could have done that as well). We specified just the directory that contains it. We can specify as many directories as we need separated by the path separator (; on windows and : on *nix). The difference between specifying a directory and specifying a jar is that when you specify a directory, all the jars as well as exploded module definitions present in that directory become available to the compiler (or to the jvm, in the case of execution), while in the other case, only the jar that you specify on the module path becomes available. It is alright to specify a directory on the module-path while learning but it is always better to specify the jars while developing a professional application. The command to run `simpleinterest` with explicitly listed jar files in the module-path looks like this:

```
c:\ocp21\moduletest>java --module-path
    modulejars\simpleinterest.jar;modulejars\calculators.jar  --module
simpleinterest
```

23.4.2 Module jar vs regular jar ☝

As you saw before, a modular jar is no different from a regular non-modular jar from in terms of its structure. The only thing extra in a modular jar is the `module-info.class` in its root folder. In fact, it is possible to use a modular jar just like a non-modular jar. If you are wondering whether you can run the main class of a modular jar file the old way, then yes, you can. For example, assuming that you have packaged the two modules into two jars as explained in the previous section and put them in `c:\ocp21\moduletest` directory, you can execute our `SimpleInterestCalculator` class with the following command:

```
c:\ocp21\moduletest>java -classpath calculators.jar;simpleinterest.jar simpleinterest.SimpleInterestCalculator
```

However, when you run a class using the `-classpath` or `-jar` option, you lose all the benefits of modules. Which means, the JVM will not enforce the access rules specified in module descriptors. In other words, the `module-info` of the module will be ignored altogether. A class from any jar can access a class from any other jar.

This is helpful when a team has not moved to the modular structure for their project but wants to use a modular jar produced by another team. It also lets a team modularize their code without impacting their non-modular users.

23.4.3 Automatic modules ☝

An important thing to remember about modules is that when you run an application

using the `module-path` option, the application can read other modules only if the application's module info has appropriate `requires` clauses for the modules that it wants to read. Furthermore, as discussed before, those modules should also have exported the relevant packages using appropriate `exports` clauses.

This poses a problem if you want to develop a module that depends on a third party non-modular jar. How will your code access classes from the non-modular jar? It is actually a very common scenario. You might want to use a third party logging framework, a JDBC driver, or a connection manager in your application. But you wouldn't want to wait until the providers of such functionality update their jars to modular jars and, obviously, you can't take it upon yourself to modularize someone else's code.

Java modules functionality provides a nice way to handle such situations. If you simply put a non-modular jar on the `module-path`, Java will consider the non-modular jar to be a module. Such a module is called "**automatic module**".

Name of an automatic module ☝

Just like regular modules an automatic module is also a **named module** but, since there is no `module-info` for this module where you can specify its name, the name of this module is created automatically by the Java Module system using the name of the jar file. For example, if you put `mysql-driver-6.0.jar` in `--module-path`, it will be loaded as an automatic module with name `mysql.driver`.

Name derivation for an automatic module is explained in detail in `java.lang.module.ModuleFinder` JavaDoc, but for the purpose of the exam, you only need to know that hyphens are converted into dots and the trailing version number (if present) and the file extension part are ignored. Here are a few examples:

`myJdbcDriver.jar` : `myJdbcDriver`
`com_xyz_utils.jar` : `com.xyz.utils`
`commons-logging-1.3.1.jar` : `commons.logging`
`commons-collections-4.5.0-M1.jar` : `commons.collections`
`my-Jdbc.Driver.jar` : `my.Jdbc.Driver`

It is also possible for a non-modular jar to specify its future module name using `Automatic-Module-Name: <module name>` entry to the jar's `MANIFEST.MF`. This is useful when a team plans to modularize their library in future.

Since there is no `module-info.class` in a non-modular jar, there is no way to specify the packages that it exports using the `exports` clauses or to specify the jars that it depends on using the `requires` clauses. For this reason, Java provides an automatic module with **two super powers**:

1. an automatic module **exports** as well as **opens** all of its packages (so that any module can "read" those packages) and
2. an automatic module is allowed to **read all exported packages** of modules that are available on the `module-path` as well as all classes available on the `classpath`.

In other words, all public classes present in the jar file loaded as an automatic module are accessible to all other modules, and all the classes of that jar are able to access all exported packages of all modules as well as all classes that are present on the `classpath`.

Let's see how these special powers come in handy while developing a new module.

If your module depends on a non-modular third party jar, you just need to add a `requires <module-name>;` clause in the module-info of your module and put that third party jar in your `--module-path`.

For example, let's say the `SimpleInterestCalculator` class in our `simpleinterest` module uses `org.apache.commons.logging.Log` class from `commons-logging-1.3.1.jar`. We need to update the module info for `simpleinterest` as follows:

```
module simpleinterest{
   requires commons.logging; //Observe that we are specifying
```

```
the module name as derived from the name of the jar file. All packages are
exported automatically by an automatic module.
}
```

and, assuming that `commons-logging-1.3.1.jar` is present in the `jars` directory, we will include it in the `--module-path` of our `javac` as well as `java` commands as follows:

`javac --module-path jars\commons-logging-1.3.1.jar --module-source-path src -d out --module simpleinterest`
and
`java --module-path jars\commons-logging-1.3.1.jar;out --module simpleinterest/simpleinterest.SimpleInterestCalculator`

Okay, so that takes care of the part where a module requires classes from a non-modular jar. What if that non-modular jar requires classes from some other non-modular jar? For example, what if a class in `commons-logging-1.3.1.jar` refers to a class in another third party jar named `log4j-api-2.23.1.jar`? Do we have to convert `log4j-api-2.23.1.jar` into an automatic module as well? No, not necessarily.

We have to create an automatic module out of a non-modular jar only if a modular jar is **statically dependent** on a class from a non-modular jar, i.e., only if a module refers to a class from the non-modular jar in its code and therefore, requires it during compilation. If an automatic module requires a class from a non-modular jar, we need to simply place the non-modular jar on the `classpath` during the execution of the module. Remember the second super power of an automatic module - an automatic module is allowed to read all classes available on the `module-path` as well as on the `classpath`. That is what makes it work.

Thus, all we need to do is to include this jar on the `classpath` in our `java` command as follows:

```
java --module-path jars\commons-logging-1.3.1.jar -classpath jars\log4j-api-
2.23.1.jar --module simpleinterest/simpleinterest.SimpleInterestCalculator
```

As you can see, an automatic module is similar to other named modules but because of its super powers, it acts as a bridge between the modular and the non-modular code. Without automatic modules, it wouldn't be possible for modular code to make use of existing non-modular code.

23.4.4 Unnamed Module ☝

All of the classes on the `classpath` are loaded by the JVM as a part of a single "**unnamed module**". Just like classes of an automatic module, classes of the unnamed module are also allowed to "read" all exported packages of all modules available on the `module-path` and all the classes available on the `classpath`. However, since this unnamed module has no name, it is not possible to include this module in the `requires` clause of any module. Therefore, classes of an unnamed module are readable to other classes that are loaded from the `classpath` and to classes in automatic modules **but not to** explicitly named modules.

23.4.5 Summary of readability of modules ☝

The following table summarizes the readability rules for the three types of modules:

| **Classes In↓ Can read→** | **Explicitly Named Modules (provided an `exports` clause exists in the module-info)** | **Automatic Modules** | **Unnamed Module** |

Explicitly named modules (provided a `requires` clause exists in the module-info)	All exported and opened packages	All packages	Cannot read any package
Automatic Modules	All exported and opened packages	All packages	All packages
Unnamed Module	All exported and opened packages	All packages	All packages

From the table, you can infer that an explicitly named module can only read those packages that have been exported or opened by another explicitly named module. It can read all packages of an automatic module but cannot read any package in the unnamed module.

On the other hand, automatic modules and the unnamed module can read all exported and opened packages from all named modules and can read all packages of automatic and unnamed modules. It is not important for the exam but good to know that even though the compiler will not be able to check if a class from the unnamed module tries to access a class of an unexported package of a module during compilation of a module, but the JVM will raise an IllegalAccessError at runtime when such access is attempted.

23.4.6 Summary of command line switches used for compilation ☝

For compiling a Java class and/or module, you need to remember the following five command line options:

1. **`--module-source-path`**: This switch is used to specify the location of the module source files. It should point to the directory that contains source module definition. In other words it should point to the parent directory of the directory where `module-info.java` of the module is stored. For example, if your module name is `moduleA`,

then the `module-info.java` for this module would be in `moduleA` directory and if `moduleA` directory exists in `src` directory, then `src` should be specified in the `--module-source-path` option, i.e., `--module-source-path src`

If `moduleA` depends on another module named `moduleB`, and if `moduleB` directory exists in `src2` directory, you can add this directory in `--module-source-path` as well, i.e., `--module-source-path src;src2`.

2. **`-d`**: This switch is **required** when you compile a module. It is used to specify the output directory. This is the directory where `javac` will generate the module and package driven directory structure and the class files for the sources. For example, if you specify `out` as the output directory, `javac` will create a directory under `out` with the same name as the name of the module and will create class files with appropriate package driven directory structure under that directory.

3. **`--module`** or **`-m`**: This switch is used when you want to compile all the source files of a particular module. This option is helpful when you want to compile all the files at once without listing any of the source files of a module individually in the command.
For example, if you have two Java files in `moduleA`, stored under `moduleA\a\A1.java` and `moduleA\a\A2.java`, you can compile both of them at the same time using the command:
`javac --module-source-path src -d out --module moduleA`
Javac will find out all the Java source files under `moduleA` and compile all of them. It will create the class files under the output directory specified in `-d` option, i.e., `out`. Thus, the `out` directory will now have two class files - `moduleA/a/A1.class` and `moduleA/a/A2.class`.

4. **`--module-path`** or **`-p`**: This option specifies the location(s) of any other module upon which the module to be compiled depends. You can specify the module directories or jar files containing the module classes here. For example, if you want to compile `moduleA` and if it depends on some other module named `abc.util`, then you can add this to your javac command like this:

```
javac --module-source-path src --module-path thirdpartymodules/abc.util.jar -d out --module moduleA
```

5. **`-classpath`** or **`-cp`** or **`--class-path`**: This option is used for compilation of non-modular code. If you are compiling regular non-modular code but that code depends on some classes, then you can put those classes/jars on the classpath using `-classpath` option.
Note: This option is not helpful for compilation of modular code because classes in modular code cannot read classes present on classpath. Modular code can only see other modular code. That is why, non-modular classes have to be converted into "automatic modules" and specified on `--module-path` as explained earlier.

23.4.7 Summary of command line switches used for execution ☝

You need to know about the following three command line switches for running a class that is contained in a module:

1. **`--module-path`** or **`-p`**: This switch tells the location of the module definition. If the module is packaged in a jar file, then you can either specify the path to the jar, or specify the path to the directory where the jar file is stored (relative to the current directory or an absolute path). For example, `--module-path c:\javatest\modulejars`
You can also specify the location where the compiled module exists in the exploded form. For example, if your module is named `abc.math.utils` and this module is stored in `c:\javatest\output`, then you can use: `--module-path c:\javatest\output`. Remember that `c:\javatest\output` directory must contain `abc.math.utils` directory and the module files (including `module-info.class`) must be present in their appropriate directory structure under `abc.math.utils` directory.

You can specify as many module locations separated by path separator (; on windows and : on Linux/MacOS) as required.

2. **`--module`** or **`-m`**: This switch is used to specify the module that you want to run. For example, if you want to run `abc.utils.Main` class of `abc.math.utils` module, you should write `--module abc.math.utils/abc.utils.Main`
If a module jar's manifest contains the `Main-Class` property, you can omit the main class from the command. For example, you can write, `--module abc.math.utils` instead of `--module abc.math.utils/abc.utils.Main`.

3. **`-classpath`** or **`--class-path`** or **`-cp`**: This switch is used while executing non-modular code. It is also used while executing a modular code to specify non-modular jars that are required by the project.
It is possible to treat modular code as non-modular by ignoring module options altogether. For example, if you want to run abc.utils.Main class using the older classpath option, you can still do it like this:
`java -classpath mathutils.jar abc.utils.Main`

23.5 Using services ☝

So far, we have created a `simpleinterest.SimpleInterestCalculator` class in the `simpleinterest` module that implements `calculators.InterestCalculator` interface defined in the `calculators` module. We have also been able to run `simpleinterest.SimpleInterestCalculator` class as a standalone program because it has an appropriate `main` method.

You can, however, notice that `SimpleInterestCalculator` implements a single

functionality or a "service", which is defined by the `InterestCalculator` interface, and that this service can be used as a black box by anybody. In fact, the whole point of creating the `InterestCalculator` interface is that any class that requires interest to be computed could use whichever `InterestCalculator` implementation was made available to it. For example, we can also create a `CompoundInterestCalculator` that implements the same interface. There could be a huge application that needs to compute interest as a part of its larger functionality. Instead of worrying about "how" the interest is computed, this application should just use whichever `InterestCalculator` is configured for use. We could configure it to use `SimpleInterestCalculator` while the mocking the UI then configure it to use the `CompoundInterestCalculator` later. We should not need to make any change in the application code to achieve this. In other words, the application code should not be tightly coupled with the actual implementation class that provides the `InterestCalculator` service but should only depend on the service definition, i.e., the interface of that service.

There are multiple ways to achieve this goal including Spring's dependency injection framework. But that is obviously not the focus of this exam. The OCP exam focuses on a less popular technique that is available in the JDK and doesn't require any third party library. The module system makes this technique the simplest of all to use. So, let's check it out.

23.5.1 Defining and using a service ☝

A service is nothing but by a well-known interface that outlines the functionality that it provides through its methods. It has to be well-known because the service is all that the client application knows about and is coupled to. Technically, a service could be a concrete class as well but usually it is an interface or an abstract class.

So, for example, `calculators.InterestCalculator` could be a service. There is nothing special that you need to do in the interface or the class to designate it as a service really. If you think it describes a service then it is a service. Large projects usually adopt a convention such as appending the word "Service" to the names of

services.

A service provider is a class that implements the service. If the service is an interface then the service provider implements that interface and if the service is a class, then the service provider may be a subclass. In our case, `simpleinterest.SimpleInterestCalculator` would be a service provider that implements the service represented by the `InterestCalculator` interface. A class must follow certain rules to be a service provider. I will list them shortly.

Now that we have a service and a service provider, all we need is a way for the consumer application that consumes the service to get hold of a service provider. Note that the consumer application doesn't know about the service providers. It only knows about the service. This is achieved in three steps:

Step 1: Tie the service and the service provider together in the `module-info` of the module that contains the service provider using the `provides-with` directive. For example:

```
module simpleinterest{
  requires calculators;
  provides calculators.InterestCalculator with
simpleinterest.SimpleInterestCalculator;
}
```

You should observe the following two points about the above `module-info`:

1. This module **requires** the `calculators` module. This is necessary because it implements the `InterestCalculator` interface, which is defined in the `calculators` module.
2. This module **does not export** the `simpleinterest` package. There is no prohibition on exporting any package from the service provider module but it is recommended to not do so to avoid any accidental dependency on it. After all, we don't want clients to be tightly coupled to the service provider, which means, we don't want any one to directly refer to the `SimpleInterestCalculator` class.

Step 2: Use the `uses` directive in the `module-info` of the consumer application that wants to use the service to declare that it uses that service. For example, an application packaged in an entirely separate module can declare that it uses the `InterestCalculator` service as follows:

```
module consumerclient{
  requires calculators;
  uses calculators.InterestCalculator;
}
```

Observe that this module also requires the `calculators` module because that is the module in which `InterestCalculator` interface, i.e., the service, is defined but does not require the `simpleinterface` module.

Step 3. Use the `java.util.ServiceLoader` class in the consumer code to get a list of service providers that provide the service it is interested in and use them! For example, the client program named `ConsumerMain` packaged in the `consumerclient` module can use the `InterestCalculator` service as follows:

```
package client;
import calculators.InterestCalculator;
import java.util.ServiceLoader;
public class ConsumerMain{
  public static void main(String[] args){
    ServiceLoader<InterestCalculator> intrstCalcs =
ServiceLoader.load(InterestCalculator.class);
    for (InterestCalculator calc : intrstCalcs) {
                System.out.println("Got calculator : "+calc);
                System.out.println(calc.calculate(100, .05, 3));
    }
  }
}
```

The `ServiceLoader.load` method fetches all service providers that provide the given service. If more than one providers are available, it is the consumer's responsibility to pick the one it wants to use. In the above code, I am just looping through all of them and invoking the `calculate` method on each of them. Since there is just one provider in this example, it doesn't matter, but ideally, the service interface should provide methods that can help the consumer decide if it really wants to use that particular service implementation or not.

Compiling and Running the consumer client ☝

Add the `provides-with` clause in the `simpleinterest` module as described in Step 1 and then compile this module using the following command:

```
C:\ocp21\moduletest>javac -d out --module-source-path src --module simpleinterest
```

Create a module named `consumerclient` under the `src` folder with `module-info.java` as described in Step 2. Create a `ConsumerMain.java` with content as listed in Step 3 above inside the `consumerclient/client` folder. The `consumerclient` module can now be compiled with the following command:

```
C:\ocp21\moduletest>javac -d out --module-source-path src --module consumerclient
```

Once compiled, it can be run using the following command:

```
C:\ocp21\moduletest>java --module-path out --module consumerclient/ConsumerMain
```

It should generate the following output:

```
Got calculator : simpleinterest.SimpleInterestCalculator@5b6f7412
15.0
```

Note that compiling the `simpleinterest` module first separately is important after updating its `module-info`. You can't expect that `javac` will automatically compile

the `simpleinterest` module if you ask it to compile the `consumerclient` module. This is because the `consumerclient` module has absolutely no coupling with the `simpleinterest` module. So, compiling the `consumerclient` module will only cause the compiler to compile the `calculators` module along with the `consumerclient` module.

Requirements for a Service Provider ☝

Since an instance of the service provider class is required to actually perform the service, the service loader architecture expects the service provider to follow a few rules so that it can acquire an instance of the service provider. These are as follows:

1. The service provider must be **public** and must not be an inner class. It can be a top level class or a nested static class.
2. The service provider should have a **public static** method named **`provider`** that takes no arguments and has a return type that is assignable to the service's interface or class. If the service provider has such a provider method, then it will be invoked whenever an instance of the service provider is needed. Note that it is not necessary that this method must return a new instance. It could just return a cached instance too.
3. If the service provider does not have a provider method, then it must have a **public** no-args constructor. It will be used to instantiate the service provider whenever needed.

In the above example, I did not need the `SimpleInterestCalculator` class to have a provider method because it had a public no-args constructor.

Well, that's all there is to using services as far as modules are concerned. The ServiceLoader architecture is quite flexible and allows packaging and discovery of services even without modules. However, I will not go into that because it is not relevent for the OCP exam.

You might have read about the need to use a "**Service Locator**"

to locate services. Service Locator is a well known design pattern that is used to separate out the logic of locating one or more services in a one place. It is quite useful when an application has multiple services, multiple providers for multiple services, or uses multiple mechanisms to locate services such as ServiceLoader, JNDI, or dependency injection. However, it is a higher level abstraction and has nothing to do with the OCP exam. As far as the OCP exam is concerned a class that uses the `ServiceLoader` and retrieves a service from it, is the consumer of that service. Whether this class acts as a service locator or uses that service itself is entirely up to the design of the application.

23.6 Migrating non-modular applications ☝

Reorganizing a non-modular application to use modules and compiling and running it using the modular options that you saw earlier is called migrating an non-modular application to a modular application. Ideally, a completely modular application is composed of only explicitly named modules, i.e., it does not contain any automatic modules or the unnamed module.

Usually, making an application modular does not involve any change in the code but since modules create impermeable boundaries between the parts of a program by enforcing strict encapsulation principles, migrating an application is not always straight forward. Migrating any non-trivial application raises the following challenges:

1. How to split the application code base into modules?
2. How to handle libraries provided by other teams of the organization that have already been modularized?
3. How to handle libraries provided by other teams of the organization that have not yet been modularized but will be modularized?
4. How to handle third party libraries that are not expected to be modularized?

The first challenge is entirely about application design and there is no official list of steps that you can just follow. But here are a few guidelines based on experience. Since application is usually already organized into packages, the easiest approach would be to just create a module per package but that actually defeats the purpose of having modules in the first place. A module is supposed to be composed of all of the functionality that is usually required together. For example, if your application uses single sign-on (where a user logs in just once and then their identity is transferred across different applications transparently), you could have a module containing all the functionality related to managing, authenticating, and authorizing a user. This module could then be used by any application that wants a user to sign in. In this case, this module will most likely contain multiple packages. However, it wouldn't be a good idea to include code related to capturing the userid/password from a web page or displaying user information in that module because that part is not something that every application will require.

Similarly, if you think that all financial calculators created by your group are usually used together, you may bundle them up in a single module even if you have organized them in different packages internally.

Since application design is a highly subjective area, it is not on the OCP exam. The OCP exam focuses squarely on the rest of the three challenges.

23.6.1 Migration strategies ☝

Since an application usually depends on code that is not managed by the same application team, it is not always possible to make an application completely modular from the start or to migrate it to be completely modular immediately. However, it is possible to start modularizing parts of the application piece by piece and make it completely modular eventually sometime in the future.

The Java team that designed the Java Platform Module System describes two strategies for migrating an application in "The State of the Module System" document, which is available at https://openjdk.org/projects/jigsaw/spec/sotms/ These

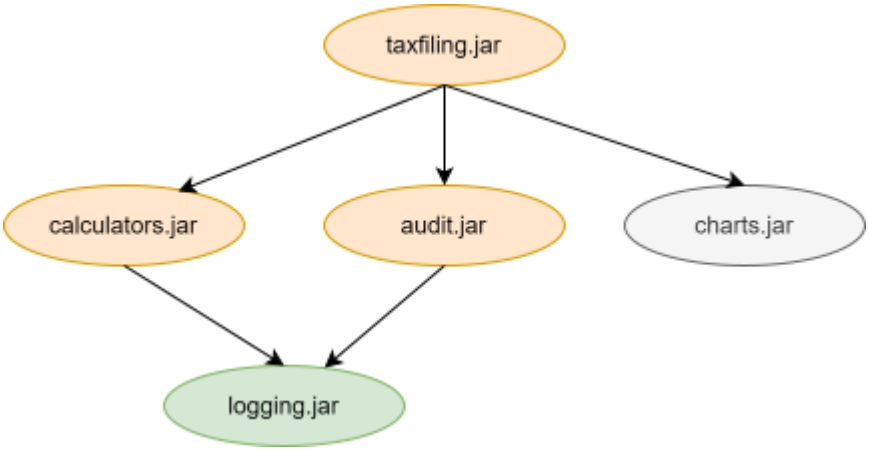
strategies are called **"Bottom up"** and **"Top down"** migration strategies. The OCP exam expects you to know both of them.

To make it easy for you to understand, I will use a hypothetical application **created by our team**, which is packaged in a regular jar file named `taxfiling.jar`. Classes in this jar use classes from three other jar files:

1. a regular jar file named `calculators.jar`, which is also produced by **our team**,
2. a regular jar file named `audit.jar` produced by another team of our same company,
3. and a regular jar named `charts.jar`, which is produced by another company.

The classes in `calculators.jar` and `audit.jar` further use an open source logging library packaged in a **modular jar** named `logging.jar`., whose module name is `logging`. So, running this application involves five jar files in total.

Based on the above information, we can draw a dependency graph for the above jar files as shown in the figure below:



Let's further assume that this application is non modular currently and is run from the command line as follows:

```
java -classpath
```

```
taxfiling.jar;calculators.jar;audit.jar;logging.jar;charts.jar
com.abc.taxfiling.Main
```

Our end goal is to make it completely modular and to be able to run it from the command line as follows:

```
java                                                    --module-path
taxfiling.jar;calculators.jar;audit.jar;logging.jar;charts.jar      --
module taxfiling/com.abc.taxfiling.Main
```

Note that you should be familiar with the `-classpath`, `--module-path`, and `--module` options by now.

23.6.2 Bottom-up Migration ☝

In the Bottom-up migration strategy, we first try to convert those parts of the application into modules that do not depend on any other non-modular part and place them on the module-path. Here, a "part" corresponds to a jar file that the application uses. We then repeat this process until we are able to convert the last part, i.e., the top most jar in the dependency graph, into a module. Let's see how this works with our hypothetical example, step by step.

Step 1:
Since it is given that `logging.jar` is already modular and doesn't depend on anything else, we can immediately put it on the module path. Hence, our command to run the application now changes to:

```
java --module-path logging.jar -classpath
taxfiling.jar;calculators.jar;audit.jar;charts.jar
com.abc.taxfiling.Main
```

Observe that the jar containing the entry point of the application, i.e.,

`taxfiling.jar`, is still on the classpath. This command works because every class that is loaded from the classpath becomes part of the unnamed module and classes on the unnamed module are allowed to read all packages from the unnamed as well as named modules.

Step 2:
We can now shift our attention to `calculators.jar`. Since it doesn't depend on any non-modular jar and is controlled by our team, it is a good candidate for migration. To convert this library into a module, we need to create a `module-info.java` with the following contents and place it into the root folder of the source code of the calculators library:

```
module calculators{
  requires logging; //since this is the name of the module packaged in logging.jar
  exports calculators; //assuming that the package name that we want to export is calculators
}
```

Since this library doesn't depend on anything else, it can be easily recompiled into a modular jar using the compilation options for compiling a module discussed before. At the end of this step, our command to run the application will be:

```
java --module-path logging.jar;calculators.jar -classpath taxfiling.jar;audit.jar;charts.jar com.abc.taxfiling.Main
```

Step 3:
As this point, we have to wait until the other team that is responsible for producing `audit.jar` modularizes it. Once the other team modularizes `audit.jar`, we can place it on module-path just like `calculators.jar`. However, we will now be stuck until the company responsible for producing `charts.jar` modularizes it (unless we use it as an "automatic" module). Let's say they modularize `charts.jar` as well and in that case, our command will become:

```
java --module-path logging.jar;calculators.jar;audit.jar;charts.jar -classpath taxfiling.jar; com.abc.taxfiling.Main
```

Step 4:
We are now in a position where we can covert `taxfiling.jar` into a module by creating the following module-info (assuming that audit and charts modules contain appropriate `exports` directives that make the packages used by taxfiling application readable):

```
module taxfiling{
  requires calculators;
  requires audit;
  requires charts;
}
```

After the recompilation and repackaging of the taxfiling application with the above module-info, our final command to run this application will be:

```
java --module-path logging.jar;calculators.jar;audit.jar;charts.jar; taxfiling.jar; --module taxfiling/com.abc.taxfiling.Main
```

That is exactly what we wanted to achieve.

You must have noticed that this process has a serious issue. We cannot modularize `taxfiling.jar` until all its dependencies are modularized and since we are not in control of all the dependencies, we are at the mercy of other teams. This is undesirable in most situations, and so, the bottom-up approach is seldom used for migrating non-trivial projects.

To summarize, in the Bottom-up approach:

1. Dependencies are migrated first and the target application jar is migrated later.
2. Every jar (or every potential module) gets moved to the module-path.

3. Nothing is left on the classpath.

23.6.3 Top-down Migration ☝

In the Top-down migration strategy, we start with converting the top most application jar into a module. While doing so, we must also place those non-modular jars on the module-path upon which the topmost module directly depends, thereby turning them into "automatic modules". Recall that an automatic module exports all of its packages and is allowed to read all exported packages from other modules. Also, recall that the module name of an automatic module is derived using the name of its jar file and is therefore, known in advance. This name is used in the module-info of the top most jar. Let's see how this will work in our case.

Step 1:
We start by creating the module-info for `taxfiling.jar`. Since it is given that it depends on `calculators.jar`, `audit.jar`, and `charts.jar`, which are not modular jars, we must use these jars as automatic modules and, in that case, their module names will be `calculators`, `audit`, and `charts`, respectively. Thus, the `module-info.java` for the `taxfiling` module will be as follows:

```
module taxfiling{
  requires calculator;
  requires audit;
  requires charts;
}
```

We can now recompile the code for the taxfiling module and create a new modular `taxfiling.jar`. Hence, our command to run the application now changes to:

```
java --module-path logging.jar;calculators.jar;audit.jar;charts.jar;taxfiling.jar --module taxfiling/com.abc.taxfiling.Main
```

Observe that the jar containing the entry point of the application, i.e., `taxfiling.jar`, is now on the module-path already. So, it looks like we are done but actually we are not. We haven't converted `calculators.jar` to a regular module yet.

Step 2:
We can follow the same process to convert `calculators.jar` into a module. We can use the following `module-info.java` for this module:

```
module calculators{
  requires logging;
  exports calculators;
}
```

Observe that we are now officially exporting the `calculators` package from the `calculators` module. In the previous step, this package was exported automatically. After recompiling and creating the new modular `calculators.jar`, we can place it on the module-path and run the application with the same command as before. Now, we are done.

Step 3:
Since we don't control the source code for other libraries, we don't try to modularize them. We are already using them as automatic modules. Once they migrate their code to modules, we can use their new jars (with appropriate changes in the module-info for the `taxfiling` and `calculators` modules if needed) but our command won't change. Note that if an automatic module depends on a third party non-modular jar, we don't need to convert that jar into an automatic module. It can remain on the classpath. We don't have that case in this example, but if `logging.jar` were not a modular jar, we would leave it on the classpath because `taxfiling.jar` doesn't depend on it directly.

As you can see we can proceed with modularization of our code in the top-down approach without waiting for other teams to finish modularization of their code. Since, with this approach all teams can work on modularizing their code base as per

their convenience, this is the preferred approach in most cases. The only issue with this approach is that we may have to edit our module-info if we get a new modular jar that has a different module-name than what we have used.

To summarize, in the Top-down approach:

1. The target application jar is migrated first and the dependencies are migrated later.
2. Not every jar (or not every potential module) gets moved to the module-path.
3. Some jars may be left on the classpath

You may see ambiguous and vaguely worded statements in a few questions about modules. Just remember the above summary and cross out the options that are obviously incorrect and then select the best option.

23.6.4 Command line options for working around the restrictions imposed by modules ☝

Ideally, a module should not depend on those packages of a library that the library developer has not intended for public access. However, in the real world, existing Java applications may inadvertently be referring classes of such packages because before modularization a library developer had no way to prevent access to internal packages. This has happened a lot with JDK's internal APIs as they were publicly accessible in the previous (prior to Java 11) unmodularized versions of the JDK. These applications will get compilation as well as runtime errors when they try to use the new modular versions of such libraries. To enable older applications use newer modular libraries, Java provides following the command line options (applicable for `javac` as well as `java`):

1. **`--add-exports module/package=other-module(,other-module)*`** : This option specifies a package to be considered as exported from its defining module to additional modules or to all unnamed modules when the value of other-module is

ALL-UNNAMED. This is useful when a module does not export a package through an exports clause in its module-info but another module has a compile time dependency on it.

2. `--add-reads module=other-module(,other-module)*` : This option specifies additional modules to be considered as required by a given module.

There are a few other options as well but you may ignore them for the purpose of the exam.
The exam does not go into the details of these options but some candidates have reported seeing these options in the exam and so, it is better to be aware of them.

23.7 Understanding the modular JDK and Tools

23.7.1 Modular JDK ☝

If you look at the size of the Java runtime library, it is hard to believe that Java was originally designed to run on even the smallest of devices. Now, thirty years later, Java has gone far beyond its goal. It has been enormously useful in building large mission critical applications as well as small desktop applications. One of the reasons for its success is the constant evolution of the features, tools, and libraries that Java comes bundled with.

Another key promise of Java was WORA, "Write Once Run Anywhere". Java came with a single set of API that you could use to write an application that would work the same on all platforms. Java achieved this feat by developing a run time environment that hid the differences in the underlying hardware and provided a common set of features.

Over the years, the types and capabilities of devices differed vastly and it become

impossible to hide over the differences in hardware capabilities of so many platforms any longer. But if you have a Java runtime for a device and if that runtime is not able to run a Java application developed for another device, then that would amount to abandoning the WORA promise. Java did finally abandon WORA when it came up with multiple versions of the runtime such as J2SE, J2EE, and J2ME. J2ME was small footprint version of the Java runtime that could run on initial generation of mobile phones. It was vastly different from J2SE and J2EE in terms of capabilities. However, the core Java library still remained the same.

By Java 8, the standard Java library, with over four thousand five hundred classes, had become a big monolithic pile of interdependent classes and packages that was impossible to split into smaller independent pieces. Even if an application used a very small part of the runtime library, it still had to be bundled with the whole library. You might have heard about a jar file named `rt.jar` that every Java installation used to comes with. This 60 MB jar file contained the complete Java SE library. If you ever wanted to distribute your Java application comprising just a single class, you would have to include this huge jar file along with it.

Modular JDK is an effort to solve this problem. It divided the JDK into several loosely coupled modules that can be combined at compile time, build time, as well as run time, into a variety of configurations as per the needs of the applications. For example, a large service side application can use the full JDK but a micro-service can be packaged with only the modules that are absolutely essential. It also allows applications that are bundled along with the runtime, to be shipped with smaller profiles, thus, increasing the reach of Java applications to more devices and platforms.

23.7.2 Organization of the modular JDK ☝

The packages, or API packages, as they are called in technical jargon, contained in the Java platform are distributed into various modules. These modules can be categorized into two broad categories - standard modules and non-standard modules. The modules that are governed by the Java Community Process (JCP) are called Standard

modules. Their names start with `java`. For example, `java.base`, `java.sql`, and `java.logging` are standard modules. All other modules that are specific to a JDK are called non-standard modules. Their names start with `jdk`. For example, `jdk.jdeps`, `jdk.rmic`, and `jdk.javadoc` are non-standard modules.

Similarly, standard packages have names starting with `java` or `javax` such as `java.lang`, `java.sql`, `javax.crypto`, and `javax.net`, and the JDK specific non-standard packages, generally, have names starting with `jdk` such as `jdk.internal.perf`, `jdk.internal.logger`, and `jdk.internal.util`. Depending on the provider of the JDK, it may have packages reflecting the company name as well such as `sun.net` or `sun.utils.resource`.

Note that a standard module may contain non-standard packages but a non-standard module does not contain any standard package.

The standard and non-standard modules are then combined to make a variety of Java configurations such as the **full Java SE Platform**, the **full JRE**, and the **full JDK**.

The Java SE Platform ☝

The Java Platform, Standard Edition (Java SE) is the **core Java platform** for general-purpose computing. It is composed only of the standard modules (i.e. whose names start with `java`) and is required to be supported by all Java implementations. You do not need to remember all the modules available in this platform but some of the most commonly used modules in this platform are `java.base`, `java.desktop`, `java.logging`, `java.sql`, `java.prefs`, and `java.net.http`. All of the programs that you have written or will write for preparing for the Java certification exam will require just this platform.

Note that this does not imply that the Java SE platform includes all of the standard modules. For example, `java.cardio` is a standard module (it is governed by the JCP) but is not a part of the Java SE Platform. Neither does it imply that this platform itself will not make use of classes from non-standard packages or modules. As I mentioned before, a standard module may contain non-standard packages. All it means is that none of the modules included in the Java SE Platform "export" any non-

standard package. In other words, if your module "requires" only the modules contained in the Java SE platform, it will depend only on the standard Java packages and will be portable to all Implementations of the Java SE Platform.

The `java.base` Module ☝

This module defines the **foundational APIs** of the Java SE platform. By foundational, we mean that it lies at the core of all Java platforms. This module doesn't require any other module but every other module depends on this module. Again, you need not remember the complete list of packages contained in this module but here are a few important packages that you should be aware of - `java.lang`, `java.lang.annotation`, `java.io`, `java.nio`, `java.net`, `java.util`, `java.util.concurrent`, `java.security`, and `java.time`.

All of the Java API classes and packages that we have used in this book belong to this module. Every class that we have written in this book depends on a class from this module. Of course, `java.lang.Object` belongs to this module, after all! But did you notice that we never wrote `requires java.base;` in the module declarations of the modules that we developed in the previous section? The reason is that since everything in Java requires packages exported from this module, the Java compiler does not require it to be specified explicitly. In other words, much like the `java.lang` package is not required to be imported in a class explicitly, the `java.base` module is not required to be specified in a `requires` clause of a module explicitly. It is always assumed to be "required".

23.7.3 New tools and command line options ☝

If you look at the JDK's bin directory (for example, `C:\Program Files\Java\jdk-21\bin` on a Windows machine), you will see several executable files. These are mostly the tools that come bundled with the JDK. These tools are used to perform various activities that we do while developing, testing, and running Java

applications. You have already used three of them - `javac.exe`, which launches the Java compiler, `java.exe`, which launches the JVM, and `jar.exe`, which creates a jar file. When you graduate to developing industrial strength Java applications, you will use several other tools such as **jdb**, **javadoc**, **javap**, and **javaw**.

With the advent of Java modules, the JDK has added new tools as well as a few new switches (i.e. command line options) to the existing tools specifically to work with Java modules. For the purpose of OCP exam, you need to be aware of what these options and tools are and what they do.

Command line options related to modules ☝

We have already discussed the `--module-path` and `--module-source-path` options in detail. These are, of course, applicable to the `java` as well the `javac` command.

`--show-module-resolution`: This new switch is applied to the `java` command to make it print module resolution information while executing a Java program. Basically, if you want to know what all modules are being referred to while running a program, this is the option to use. For example, check out the output produced by Java if you use this option while executing our `simpleinterest` module.

```
C:\ocp21\moduletest\>java --show-module-resolution --module-path out --module
simpleinterest/simpleinterest.SimpleInterestCalculator
```

Output:

```
root simpleinterest file:///C:/ocp21/moduletest/out/simpleinterest/
simpleinterest requires calculators
file:///C://ocp21/moduletest/out/calculators/
java.base binds jdk.jdeps jrt:/jdk.jdeps
...
```

```
... omitting several such lines
...
java.rmi requires java.logging jrt:/java.logging
10.0
```

The first line of the output shows that the root module is `simpleinterest` (because this the module Java is about to execute) and that it is loaded from `C:/ocp21/moduletest/out/simpleinterest/` .
The second line shows that `simpleinterest` requires the `calculators` module, which is loaded from `C:/ocp21/moduletest/out/calculators` .
We know that `calculators` doesn't require any other module. However, every module requires the `java.base` module. So, the rest of the lines show the details of the loading of the `java.base` module.
The last line is the output from our `SimpleInterestCalculator` 's `main` method.

Note that the `show-module-resolution` option shows module level details only. It doesn't tell you which package or which class from a module is actually required.

jdeps ☝

`Jdeps` is a new tool specifically designed to help you deal with modules without executing them. Using this tool, you can find out module, package, as well as class level dependencies of a given module. It can analyze an exploded module or a module jar. Let's run it on our `simpleinterest` module and see what it produces.

```
c:\ocp21\moduletest\>jdeps --module-path out --module simpleinterest
```

Output:

```
[file:///C:/ocp21/moduletest/out/simpleinterest/]
   requires calculators
   requires mandated java.base (@11.0.2)
simpleinterest → calculators
```

```
simpleinterest → java.base
   simpleinterest          → calculators    calculators
   simpleinterest          → java.io        java.base
   simpleinterest          → java.lang      java.base
```

It shows that `simpleinterest` uses `calculators` as well as `java.base` modules. It also shows that `simpleinterest` depends on the `calculators` package from the `calculators` module and `java.io` and `java.lang` packages from the `java.base` module.

If you want to see class level dependencies, you can try the `-verbose:class` option:

```
c:\ocp21\moduletest\>jdeps -verbose:class --module-path out --module
simpleinterest
```

Output:

```
[file:///C:/ocp21/moduletest/out/simpleinterest/]
requires calculators
   requires mandated java.base (@11.0.2)
simpleinterest → calculators
simpleinterest → java.base
   simpleinterest.SimpleInterestCalculator    → calculators.InterestCalculator
calculators
   simpleinterest.SimpleInterestCalculator    → java.io.PrintStream
java.base
   simpleinterest.SimpleInterestCalculator    → java.lang.Object
java.base
   simpleinterest.SimpleInterestCalculator    → java.lang.String
java.base
   simpleinterest.SimpleInterestCalculator    → java.lang.System
java.base
```

Jdeps is a powerful tool with several options but for the OCP exam, the information provided above is sufficient.

jmod ☝

Java 9 also introduces a new way to package Java code into "jmod" files. This packaging method is used when you need to package artifacts such as native code that can't be put in jar files. However, a jmod file cannot be executed and is therefore, only used at compile time and link time. It is not a replacement for jar files. Don't worry, you don't need to go in the details of jmod files. But you may still see the `jmod` command mentioned in an exam question. For that, you should remember the basic syntax of the `jmod` command:

```
jmod (create|extract|list|describe|hash) [options] jmod-file
```

The first argument to `jmod` is its operation mode. It has to be one of five - `create`, `extract`, `list`, `describe`, and `hash`.

1. `create` - Creates a new JMOD archive file
2. `extract` - Extracts all the files from the JMOD archive file
3. `describe` - Prints the module details
4. `list` - Prints the names of all the entries, i.e., files contained in the jmod file
5. `hash` - Determines leaf modules and records the hashes of the dependencies that directly and indirectly require them.

Out of the above, only the `describe` mode is related to modules. An important difference between the `jmod` command and `java/javac` commands is that hyphen (, i.e., `-`) is not used to specify mode.

JImage ☝

Normally, running a Java application requires the presence of the standard JRE, which includes all the standard modules and packages and is therefore, quite large in terms of disk size. The modularized JDK introduced with Java 9 allows you to create a customized JRE image containing just the parts that are required for your application thereby reducing the overall size by a lot. Such a custom JRE image is packaged in a

`.jimage` file. A JImage file can be created and managed using tools such as `jlink` and `jimage`, which are available in the JDK.

It is not clear how deep the exam expects you to go into the creation of a JImage file but based on feedback from the recent OCP Java 21 exam takers, you should at least know the following points:

1. Creation of a JImage does not require an application to be modular because it is meant for creating customized JREs by cherry picking modules from the standard JRE. Customized JRE images can be created for modular as well as non-modular applications.
2. Since a JImage is basically a customized JRE, it is not platform independent (i.e. it is not "cross platform"). A JImage is created for a specific platform. Note that a Java application is platform independent but a JRE is not.
3. A JImage may include all sorts of files including binaries as well as class files from the application as well as from the JRE.

23.8 Exercise ☝

1. Create a module named `compoundinterest` containing a class named `CompoundInterestCalculator` similar to the `simpleinterest` module described in this chapter.
2. Compile the source code of `compoundinterest` module with and without using the `--module` switch. Use `-d` option to direct the output to your output directory.
3. Run the `CompoundInterestCalculator` class with and without using the `--module-path` switch.
4. Enhance your module by making the `CompoundInterestCalculator` class implement the `InterestCalculator` interface of the `calculators` module.
5. Package `compoundinterest` module into a jar and run this module using the `--module-path` switch.
6. Enhance the `compoundinterest` module to declare that it implements the

6. `InterestCalculator` service. Include this package while running the `ConsumerMain` program and observe the output.
7. Enhance the `InterestCalculator` interface to include another method that can be used to distinguish between different service providers and use this method in `ConsumerMain` to pick one of the two service providers.
8. Create a module finance with a class `TestClass`. It should have a main method that computes simple interest as well as compound interest. Make appropriate modifications to `simpleinterest` and `compundinterest` modules for this purpose.
9. Create and export a service interface named `MessagingService` with a dummy method named `sendMessage` from `messaging` module. Provide two implementations of this service and package them in two different modules. Use both the implementations of `MessagingService` from another module.